

My Simple-Local-Dynamic RAG Journey

From Documents → Retrieval → LLM Answer

What is RAG?

Retrieval-Augmented Generation (RAG) combines information retrieval with an LLM. Instead of asking the LLM to answer only from its general knowledge, we first retrieve relevant information from our own documents and provide it as context.

Our project uses a simple local RAG pipeline:

Documents → Chunking → Embeddings → FAISS → Retriever → Prompt → Ollama → Answer

The first step is collecting documents that the system should know about. Our application dynamically reads files from the **data/** folder.

Supported formats:

PDF → PyPDFLoader

DOCX → Docx2txtLoader

TXT → TextLoader

The important idea is: **we prepare the external knowledge before asking the LLM anything.**

1.Chunking

Large documents are split into smaller pieces called **chunks**. We use

RecursiveCharacterTextSplitter with a chunk size of 500 and an overlap of 50.

Why? If chunks are too large, retrieval may bring unnecessary information. If they are too small, useful context can be lost. Good chunking improves retrieval quality.

Chunking flow:

Large Document → Chunk 1 → Chunk 2 → Chunk 3 → ...

2.Embeddings

An embedding converts text into a numerical vector that represents its semantic meaning. This allows the system to compare the meaning of a user's question with the meaning of document chunks.

We use the Sentence Transformer model **all-MiniLM-L6-v2**. It is lightweight, runs locally, and is suitable for our learning project.

Example:

"How many casual leaves are provided?" → [0.12, -0.44, 0.91, ...]

The same embedding model is used for both document chunks and the user's query so their vectors can be compared.

3.FAISS Vector Store

FAISS is used to store and search the generated vectors. Our project uses **IndexFlatL2** for similarity search.

With LangChain, we create the vector store using:

```
FAISS.from_documents(documents=chunks, embedding=embeddings)
```

Conceptually:

Chunks → Embeddings → FAISS / IndexFlatL2 → Searchable Vector Store

Unlike a traditional database that mainly searches structured values or exact text, a vector store allows us to search based on **semantic similarity**.

4.Retriever

The Retriever is responsible for getting the most relevant chunks from the vector store for a user's question.

Our configuration retrieves the top 3 relevant chunks:

```
retriever = vector_store.as_retriever(  
search_type="similarity",  
search_kwargs={"k": 3})
```

Flow:

User Question → Query Embedding → FAISS Similarity Search → Top 3 Relevant Chunks

FAISS stores/searches the vectors; the Retriever provides a clean interface for retrieving relevant documents.

5.Prompt + LLM Generation

After retrieval, we have two important things: the **user's question** and the **retrieved context**. We combine them into a prompt and send it to the LLM.

Our prompt instructs the model to answer using the provided context and not invent information when the answer is unavailable.

Generation flow:

Retrieved Chunks + User Question → PromptTemplate → ChatOllama → Llama 3.2:1b → Final Answer

We use **Ollama** because it allows the LLM to run locally, which is useful for learning without depending on a paid API.

We set **temperature=0** because we want more consistent and less creative responses for a document-grounded RAG application.

Our Complete Pipeline

